

Use case development

I started this project with two raw data sources: PFF scouting grades and tracking data recorded at 10 Hz.

```
archive/pffScoutingData.csv
# 188,254 rows × 15 columns, one row per player per play

archive/tracking_parquets/week1-8.parquet
# x, y, s, a, o, dir at 10 Hz
```

At the beginning, looking at the data and researching online I had some questions around pass protection, and a feeling that the tracking data might contain more information than I was currently getting out of it. The first thing that really shaped the direction of the project came from the PFF data itself.

1. The structure of the PFF data gave me the first clue after running my EDA

I discovered the 188,254 rows fall into five player roles:

- Coverage — 57,765
- Pass Block — 46,057
- Pass Route — 39,513
- Pass Rush — 36,362
- Pass/QB — 8,557

What stood out in the EDA was how different those roles looked in terms of missingness:

- Each role has its own data signature: columns that are relevant to that role are populated at 100%, while columns belonging to the other roles are essentially 0% filled. At first, this looks like a straightforward data-cleaning issue. But I started wondering whether the same separation would appear in the tracking data. In other words:

If the PFF data can tell the roles apart so clearly, can the player's movement tell them apart too? can we use it as a proxy to discover a hidden tactical shift during games?

That became the first thing I wanted to test.

2. First test: can movement tell me what role a player is performing?

- `player_role` multiclass classification (My hypothesis D link to notebook D)

For the first model, I am not trying to predict pressure, blocking, or anything else about the play outcome.

I am starting with a simpler question:

- What role was this player performing, based only on movement?

I use every player-play row here, so the dataset is the full 188,254 rows across all five roles.

The features are deliberately based on the player's own movement:

- speed
- acceleration
- orientation variability
- direction variability
- displacement

Here I am not using PFF outcome information as features. The target is simply `pff_role`.

- I evaluate the model with Macro-F1 because this is a five-class problem and I want the performance of the smaller roles to matter rather than allowing the larger classes to dominate the metric.
 - This is also why I built this problem first.
 - If movement does not separate the roles at all, that would be a warning sign for the problems that follow. I would have to be much more cautious about assuming that role-specific movement patterns contain useful information.
 - If it does separate them, then I have some evidence that the tracking data is capturing meaningful behavioral differences between roles.
 - That gives me a reason to move from the general role question into pass protection specifically.

3. Once I had that baseline question, I moved to the blocker

- A — `pressure_allowed` — binary classification

The next question is more directly about pass protection:

Will this blocker get beaten?

- The Pass Block rows give me a useful starting point because `pff_hitAllowed` , `pff_hurryAllowed` , and `pff_sackAllowed` are filled for 100% of the pass-block assignments. So every assignment has a recorded outcome.
- I turn those into one binary target:

```
pressure_allowed = 1  
if any of hit / hurry / sack was allowed
```

Each row represents one blocker ↔ one rusher on one play, giving me approximately 44,402 rows.

- What I use as features

Here I want the model to see the interaction between the blocker and the rusher rather than just one player's movement.

The features include:

- player position
- blocking technique
- backfield flag
- separation at snap
- closest approach
- closing velocity
- speed and acceleration for both players

The exploratory analysis already suggests that there is real signal in this problem.

For example:

- PU technique has a 17.4% pressure rate versus 1.3% for CL.
- HB-L blockers are beaten 14.2% of the time versus 4.3% for centers.
- When pressure is allowed, the rusher's closest approach is 0.655 yards versus 0.829 yards on clean blocks.

So I am not starting from the assumption that the model will discover something from nothing. The EDA is already showing relationships that make the prediction problem plausible.

- The imbalance in that dataset is important as well:

Only 6.6% of the rows are positive.

That is about a 14.1:1 class imbalance, which means accuracy would be a poor way to judge the model. A model could look "accurate" while mostly predicting that nothing happens.

So I use: `class_weight='balanced'` and evaluate with `PR-AUC` rather than accuracy.

- I also kept an eyeout for leakage
 - Three fields directly define the target:
 - `pff_hitAllowed`
 - `pff_hurryAllowed`
 - `pff_sackAllowed`

Those have to be removed from the feature set and I also remove `separation_at_end`, because that is information from the end of the play and therefore comes after the outcome has had time to develop.

- The split is by game, not by row

I split by `gameId`.

The reason is that rows from the same game are not independent. They can contain the same players, the same scheme, and the same game context.

A random row-level split could therefore put highly correlated examples into both train and test and make the model look better than it really is.

4. Then I looked at the same problem from the rusher's side

- B — `pressure_generated` — also a binary classification

The obvious next question is:

Will this rusher win?

It sounds like I could just take Problem A and reverse it, but there is an important problem with doing that.

The field `pff_nflIdBlockedPlayer` identifies one opponent from the blocker's side, but that is not enough to represent every rush situation.

- It breaks down for things like:
 - double teams
 - free rushers
 - situations where the actual spatial interaction does not match a simple one-blocker-to-one-rusher assignment

So instead of assuming an assignment, I use the tracking data itself.

For every pass rusher, I look at the physically nearest blocker frame by frame.

That gives me a relationship based on what is actually happening on the field rather than assuming that one PFF assignment field completely describes the interaction.

This works for one-on-one situations, double teams, and unblocked rushers.

Each row here is one pass rusher on one play, giving me 36,362 rows.

- Features

The feature set focuses on the rusher and the nearest blocker:

- rusher speed
 - rusher acceleration
 - rusher orientation
 - nearest blocker distance, including minimum and mean distance
 - closing velocity on that blocker
 - rusher position
- Target

- The target is:

```
pressure_generated = 1  
if any of hit / hurry / sack was recorded
```

Again, I remove the direct outcome variables:

- pff_hit
- pff_hurry
- pff_sack

The metric for this problem is also PR-AUC.

The important difference from Problem A is really the way I define the interaction. I am deliberately using spatial proximity instead of relying on a single assignment field because I want the model to work across different rush structures.

5. After looking at outcomes, I wanted to understand the technique itself

- C — `block_type` — multiclass classification

The next question is different:

Can the blocker's own movement tell me which technique they used?

- Here I do not want rusher information. I want to know how much of the blocking technique is actually visible in the blocker's movement.

The class distribution makes this difficult from the beginning.

PP is the dominant class with 24,689 rows.

Other techniques are extremely rare:

- CH — 57 rows
- SR — 109 rows

So techniques with fewer than 150 occurrences are collapsed into an `OTHER` class.

That gives the model a more realistic classification problem instead of asking it to learn rare classes from almost no examples.

- Features

I use blocker movement only:

- blocker speed
- blocker acceleration
- lateral displacement
- forward displacement
- orientation variability

There are no rusher columns in this model.

That restriction is intentional. I want to test whether technique is visible in the blocker's movement independently of the opponent context.

- Target

The target is:

```
pff_blockType
```

with rare classes collapsed into `OTHER`.

- What I expect from this model

- I expect the Macro-F1 to be approximately 0.31 and I do not consider that a failure. In fact, that is part of the finding. from literature a lot of blocking technique is determined before the snap by the play call and formation. Post-snap movement therefore should not be expected to perfectly reveal the original technique. So the useful interpretation may not be “this model identifies technique perfectly.”
 - It may be more useful as a **deviation detector**: identifying movement that looks unusual for the technique being performed.

6. Proposed follow-up (not implemented): the binary pressure label is coarse

Working through Problem A, I noticed the binary label throws away useful information. For example, these two blocks can both end up as:

```
pressure_allowed = 0
```

even though they are very different:

- in one case the rusher gets within 0.4 yards of the blocker
- in the other case the rusher never gets particularly close

That gap is the motivation for a sixth question I have **scoped but not built**:

How good was this block, rather than simply whether it failed?

Status: proposed next step, no notebook exists for this yet. I'm including the design here because it's the most natural extension of Problem A, not because it's part of the delivered work.

The actual PFF 0–100 grades are not available in the dataset, so the plan would be a transparent, constructed 0–100 proxy:

```
score = 100

score -= 60  if sack allowed
score -= 40  if hit allowed
score -= 20  if hurry allowed
score -= 0–20 on clean blocks, scaled according to how close the rusher got
              (< 1.5 yards means a larger deduction)
```

The row population and features would reuse Problem A's — same population, same tracking-derived interaction features — with the target replaced by this constructed score, evaluated with R^2 and RMSE as a regression problem.

The important distinction if this gets built: the target would be transparent and constructed, meant to preserve more information about the block than the binary pressure label allows — not a claim of reproducing the actual (unavailable) PFF grade.

7. Then I ran into a different problem: most blocks never “fail”

- E — `time_to_pressure` — survival analysis

The last question came from looking at the outcome distribution more carefully.

93.4% of blocks did not fail.

The play ended before the block broke down.

That creates a problem if I treat this as an ordinary classification task.

If I throw those observations away, I lose most of the data.

If I label them as simply “survived,” I am implicitly treating “survived this play” as if it means “would survive indefinitely.”

Both interpretations are wrong.

That is why I moved to survival analysis.

The question becomes:

How long does the block last before it breaks?

- Why survival analysis fits the data

For a block that fails, I have a duration and an event.

For a block that holds until the play ends, I have a right-censored observation. I know that the block survived at least as long as the play lasted, but I do not know what would have happened after the observation ended.

That is exactly the kind of situation survival analysis is designed for.

- Row definition

The row population is the same as Problem A.

- Duration

I use:

```
window_duration_s
```

PFF does not provide a per-frame timestamp telling me exactly when the pressure event occurred.

So I cannot measure the true time-to-pressure directly.

Instead, `window_duration_s` is used as the duration proxy for all rows.

- Event

The event is:

```
pressure_allowed
```

- Features

I start from the Problem A features, but remove two things.

First, I remove:

```
separation_at_end
```

because that is post-outcome information.

Second, I remove:

```
window_duration_s
```

because it is the duration itself. Using it as a covariate would be circular.

- Models

I use two models here:

1. Kaplan-Meier survival curve
2. Cox Proportional Hazards model

The Cox model gives me a way to look at the hazard ratio associated with each feature.

- Metric

The main metric is the C-index.

This changes the question slightly compared with the earlier models.

Instead of only asking:

Did the block fail?

I can now ask:

How does the risk of failure change over the time that the block is being observed?

8. Where the use cases leave me

At this point, the five delivered use cases are giving me different views of the same underlying tracking data.

The first model asks whether movement can identify the role.

The next two ask whether that movement and interaction context can predict pressure from either side of the matchup.

The block-type model asks whether technique itself is visible in movement.

The survival model handles the fact that most observations end before a failure event occurs.

The proposed block-quality regression (\$6, not yet built) would add a sixth view that keeps more nuance than a simple binary outcome.

So I am not trying to make five versions of the same model.

I am using five different formulations — with a sixth scoped as a follow-up — to answer different questions about the same pass-protection behavior.

9. Notebook structure

The notebooks follow the investigation rather than trying to hide the work behind one shared pipeline.

Notebook	Problem
01_eda.ipynb	Exploratory analysis $\diamond$ validates all use cases
02_model_player_role.ipynb	D
03_model_pressure_allowed.ipynb	A
04_model_pressure_generated.ipynb	B
05_model_block_type.ipynb	C
06_survival_time_to_pressure.ipynb	E

The EDA notebook is where I validate whether the five delivered questions (plus the proposed block-quality follow-up from §6) are actually supported by the raw data.

Each modeling notebook then starts from the raw data, engineers its own features, defines its own target, and trains its own model.

That is intentional. I want to be able to trace each result all the way back to the raw files and understand exactly what the model was allowed to see.

10. What is my vision at the end or more how can I convert the models into a potential revenue stream:

In my mind i think these models can form a basic Pass Protection Intelligence System which will ofcourse need more refinements to be able to sell but this is a good PoC. The name fits because the system is not one giant model. It is a collection of different signals and perspectives that only become useful when I put them together.

Useful links

- [Dictionary of terminologies and columns](#)
- [GeneralKnowledge](#)